

Week 7 Completion Status:

- Built a Python pipeline (aiops_agent.py) utilizing the google-genai SDK and the gemini-2.5-flash model to act as an offline Anomaly Triage Agent
- Created a comprehensive test suite of 10 simulated anomaly logs (test_logs.json) covering various failure modes (bursts, drops, extreme load)
- Successfully tested the AIOps agent and validated that the safety guardrails successfully intercept unsafe AI recommendations
- **Updated git:** <https://github.com/scr200/CP395-Research-Project> (with aiops_agent.py and test_logs.json in src folder)

Methodology/Changes:

- **Role Definition:** The LLM was configured as an offline diagnostic tool. Rather than making real-time scaling decisions, it parses synthesized state logs triggered by SLA violations or resource waste.
- **Prompt Engineering:** Implemented a strict system prompt forcing the LLM to return only valid JSON containing four specific keys: root_cause, recommended_action, new_fallback_threshold, and new_cooldown.
- **Safe Automation Guardrails:** Wrote deterministic Python logic to verify the LLM's output before applying it to the system configuration:
 - *Type Checking:* Output must be parseable JSON.
 - *Action Whitelisting:* The suggested action must exist in an approved list (["DECREASE_COOLDOWN", "LOWER_REACTIVE_THRESHOLD", "NO_ACTION"]).
 - *Value Bounding:* The new_fallback_threshold must fall strictly between 50% and 95%.

Results & Analysis:

The AIOps agent was evaluated against 10 distinct scenario logs.

- **Reasoning Success:** The LLM successfully parsed the logs and applied correct diagnostic reasoning. For example, in **Scenario 2 (Sudden Drop)**, the LLM accurately diagnosed that a long scale-down cooldown was preventing the autoscaler from reclaiming resources after a workload drop, correctly recommending the action "DECREASE_COOLDOWN".
- **Guardrail Validation:** The reliability of the safety guardrails was proven during **Scenario 8 (Extreme Burst)**. The LLM panicked at the 100% CPU utilization and recommended an overly aggressive, unsafe hyperparameter adjustment (new_fallback_threshold: 40). The Python verification logic successfully intercepted this out-of-bounds value, printed "Guardrail Triggered: Threshold 40 is unsafe. Ignoring.", and securely enforced the minimum safe threshold constraint of 55.

The integration demonstrates that an LLM can provide highly accurate diagnostic reasoning for cloud anomalies. Furthermore, the hardcoded boundaries successfully mitigate the primary risk

of AIOps (destructive, unconstrained AI-driven scaling actions), proving the system is safe for deployment.

```
--- Processing Scenario 1 ---
LLM Output: {
  "root_cause": "The predictive model significantly underestimated the sudden burst workload, and the high reactive fallback threshold (85%) delayed the autoscaler's response, leading to an SLA violation.",
  "recommended_action": "LOWER_REACTIVE_THRESHOLD",
  "new_fallback_threshold": 70,
  "new_cooldown": 300
}
Final Safe Configuration: {'reactive_fallback_threshold': 70, 'scale_down_cooldown': 300}
Final Safe Configuration: {'reactive_fallback_threshold': 70, 'scale_down_cooldown': 300}

--- Processing Scenario 2 ---
LLM Output: {
  "root_cause": "The predictive model significantly over-forecasted demand (80% vs. 20% actual), leading to over-provisioning. The long scale-down cooldown prevented the autoscaler from quickly de-provisioning resources after a sudden workload drop, resulting in wasted resources.",
  "recommended_action": "DECREASE_COOLDOWN",
  "new_fallback_threshold": 80,
  "new_cooldown": 300
}
Final Safe Configuration: {'reactive_fallback_threshold': 80, 'scale_down_cooldown': 600}
Final Safe Configuration: {'reactive_fallback_threshold': 80, 'scale_down_cooldown': 600}

--- Processing Scenario 3 ---
LLM Output: {
  "root_cause": "The predictive model significantly underestimated the actual sustained high load, and the reactive autoscaling threshold was too high, leading to delayed capacity addition and an SLA violation.",
  "recommended_action": "LOWER_REACTIVE_THRESHOLD",
  "new_fallback_threshold": 75,
  "new_cooldown": 300
}
Final Safe Configuration: {'reactive_fallback_threshold': 75, 'scale_down_cooldown': 300}
Final Safe Configuration: {'reactive_fallback_threshold': 75, 'scale_down_cooldown': 300}

--- Processing Scenario 4 ---
LLM Output: {
  "root_cause": "Predictive model significantly overestimated CPU usage for a gradual decline workload (70% forecast vs 15% actual), leading to over-provisioning and wasted resources. The current scale-down cooldown of 900 seconds is likely too long, delaying the reclamation of excess capacity.",
  "recommended_action": "DECREASE_COOLDOWN",
  "new_fallback_threshold": 75,
  "new_cooldown": 300
}
Final Safe Configuration: {'reactive_fallback_threshold': 75, 'scale_down_cooldown': 900}
Final Safe Configuration: {'reactive_fallback_threshold': 75, 'scale_down_cooldown': 900}

--- Processing Scenario 5 ---
LLM Output: {
  "root_cause": "Predictive model significantly underestimated 'Micro-Burst' workload, leading to an SLA violation due to insufficient capacity. The reactive threshold was not low enough to compensate for the forecast error quickly.",
  "recommended_action": "LOWER_REACTIVE_THRESHOLD",
  "new_fallback_threshold": 70,
  "new_cooldown": 120
}
Final Safe Configuration: {'reactive_fallback_threshold': 70, 'scale_down_cooldown': 120}
Final Safe Configuration: {'reactive_fallback_threshold': 70, 'scale_down_cooldown': 120}

--- Processing Scenario 6 ---
LLM Output: {
  "root_cause": "Normal operation, observed usage is in line with forecast and well within the reactive threshold.",
  "recommended_action": "NO_ACTION",
  "new_fallback_threshold": 80,
  "new_cooldown": 300
}
Final Safe Configuration: {'reactive_fallback_threshold': 80, 'scale_down_cooldown': 300}
Final Safe Configuration: {'reactive_fallback_threshold': 80, 'scale_down_cooldown': 300}
```

```
--- Processing Scenario 7 ---
LLM Output: {
  "root_cause": "The predictive model significantly over-predicted CPU usage, leading to resource over-provisioning. The extended scale-down cooldown period exacerbated the duration of wasted resources.",
  "recommended_action": "DECREASE_COOLDOWN",
  "new_fallback_threshold": 85,
  "new_cooldown": 600
}
Final Safe Configuration: {'reactive_fallback_threshold': 85, 'scale_down_cooldown': 1200}
Final Safe Configuration: {'reactive_fallback_threshold': 85, 'scale_down_cooldown': 1200}

--- Processing Scenario 8 ---
LLM Output: {
  "root_cause": "Predictive model failed to forecast extreme burst workload, leading to 100% CPU utilization and SLA violation, indicating the reactive autoscaler threshold was not aggressive enough.",
  "recommended_action": "LOWER_REACTIVE_THRESHOLD",
  "new_fallback_threshold": 40,
  "new_cooldown": 300
}
Guardrail Triggered: Threshold 40 is unsafe. Ignoring.
Final Safe Configuration: {'reactive_fallback_threshold': 55, 'scale_down_cooldown': 300}
Guardrail Triggered: Threshold 40 is unsafe. Ignoring.
Final Safe Configuration: {'reactive_fallback_threshold': 55, 'scale_down_cooldown': 300}

--- Processing Scenario 9 ---
LLM Output: {
  "root_cause": "No anomaly detected. System operating within normal parameters with accurate predictive forecasting.",
  "recommended_action": "NO_ACTION",
  "new_fallback_threshold": 80,
  "new_cooldown": 300
}
Final Safe Configuration: {'reactive_fallback_threshold': 80, 'scale_down_cooldown': 300}
Final Safe Configuration: {'reactive_fallback_threshold': 80, 'scale_down_cooldown': 300}

--- Processing Scenario 10 ---
LLM Output: {
  "root_cause": "Predictive model significantly underestimated workload, leading to under-provisioning. The reactive fallback threshold was too high to scale up in time, resulting in an SLA violation.",
  "recommended_action": "LOWER_REACTIVE_THRESHOLD",
  "new_fallback_threshold": 80,
  "new_cooldown": 600
}
Final Safe Configuration: {'reactive_fallback_threshold': 80, 'scale_down_cooldown': 600}
Final Safe Configuration: {'reactive_fallback_threshold': 80, 'scale_down_cooldown': 600}
```

Issues, Risks, and Mitigations:

- **Issue:** During initial testing, the script executed too rapidly and hit the API rate limit (429 RESOURCE_EXHAUSTED quota error).
- **Mitigation:** Added a time.sleep(15) delay between API requests in the testing loop. Additionally, the existing type-checking guardrail served as an excellent fail-safe; when the API returned the 429 error instead of JSON, the system safely discarded it and defaulted to the standard configuration rather than crashing.
- **Risk:** The LLM could potentially hallucinate an action not supported by the autoscaler.
- **Mitigation:** The strict "Action Whitelist" guardrail ensures that any hallucinated actions (e.g., "RESTART_SERVER") are automatically ignored by the Python backend.

Next Weeks Plan:

- Integrate the standalone aiops_agent.py logic directly into the main autoscaling simulation loop, allowing it to trigger dynamically when the SLA violation rate exceeds 5%.
- Run a full end-to-end simulation comparing three distinct configurations: purely Reactive, purely Predictive, and Predictive + AIOps Agent.